



**정보보호**

**제품군** 

**실습훈련**

기초 ② 시스템 모의해킹



## 기초 ②

# Buffer Overflow

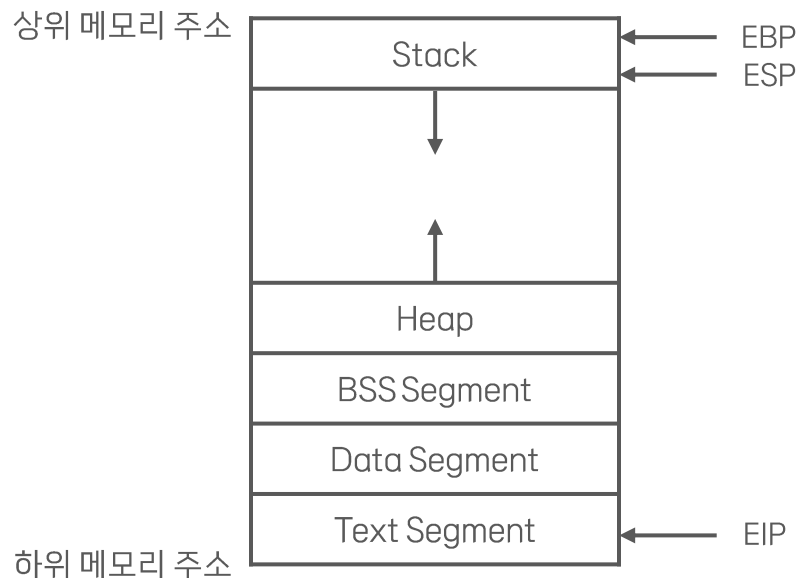
- Buffer Overflow 개요
- Stack Buffer Overflow
- Heap Buffer Overflow
- Buffer Overflow 대응 방안

# [기초2] Buffer Overflow – Buffer Overflow 개요

## 메모리 기본 구조

### ■ 메모리 구성 요소

- Stack : 함수 인자 값, 함수 내 지역 변수, 함수의 반환 주소등이 저장되는 영역
- Heap : 프로그래머의 필요에 따라 동적 메모리 호출에 의해 할당 되는 메모리 영역
- BSS 세그먼트 : 프로그램을 실행할 때 0이나 NULL 포인터로 초기화 되는 영역 (초기화 되지 않은 변수 저장)
- Data 세그먼트 : 전역 변수와 정적 변수가 저장되어 있음
- Text 세그먼트 : 프로그램 코드와 상수가 정의되어 있음, 읽기만 가능



# [기초2] Buffer Overflow – Buffer Overflow 개요

## 포인터 레지스터

### ■ 레지스터

- 처리 중인 데이터나 처리 결과를 임시 보관하는 CPU 안의 기억 장치

### ■ 포인터 레지스터

- 프로그램의 실행 과정에서 사용하는 주요 메모리 주소 값을 저장
- EBP, ESP, EIP 가 존재

# [기초2] Buffer Overflow – Buffer Overflow 개요

## 포인터 레지스터

### ■ EBP (Extended Base Pointer Register)

- Stack 세그먼트에서 현재 호출해서 사용하는 함수의 시작 주소 값을 저장
- ESP 레지스터와 함께 써서 스택 프레임을 형성
- 함수가 호출될 때 ESP를 EBP에 저장하고 있다가 모든 루틴이 끝나고 함수를 리턴하면 ESP 값을 돌려주어 스택 형성에 문제가 발생하지 않도록 함

### ■ ESP (Extended Stack Pointer Register)

- 현재 스택 영역에서 가장 하위 주소를 저장

### ■ EIP (Extended Instruction Pointer Register)

- 다음에 실행될 명령의 오프셋을 포함
- 현재 실행 중인 Code 세그먼트에 속한 현재 명령을 가리킴

# [기초2] Buffer Overflow – Buffer Overflow 개요

## 스택 프레임 (Stack Frame)

### ■ 스택 프레임 (Stack Frame)

- 스택 Segment 내부의 단위이며 함수가 호출될 때 생성되고 반환될 때 해제됨
- 함수의 스택 영역을 구분하기 위해 생성되는 공간
- 스택 프레임 영역을 할당하기 위해 함수 프로로그와 함수 에필로그가 수행됨

### ■ 함수 프로로그 (Stack Prologue)

- 함수가 호출되면 함수에 레지스터를 설정

|    |               |
|----|---------------|
| 1: | push ebp      |
| 2: | move ebp, esp |

### ■ 함수 에필로그 (Stack Epilogue)

- 함수 종료 시 레지스터를 복원

|    |       |                                          |
|----|-------|------------------------------------------|
| 1: | leave | // leave 내부 과정 (move esp, ebp / pop ebp) |
| 2: | ret   | // ret 내부 과정 (pop eip / jmp eip)         |

# [기초2] Buffer Overflow – Buffer Overflow 개요

## Buffer Overflow

### ■ Buffer

- 데이터를 일시적으로 보관하는 메모리 영역
- 정해진 일정한 크기를 가짐

### ■ Buffer Overflow

- Buffer에 할당된 용량보다 더 많은 입력이 위치하면 다른 정보를 변경할 수 있는 조건에서 발생하며, 공격자는 이러한 조건을 이용하여 코드를 삽입
- Stack Buffer Overflow와 Heap Buffer Overflow로 구분됨

# [기초2] Buffer Overflow – Buffer Overflow 개요

## Buffer Overflow 공격 원리

### ■ 예제 코드

bof.c

```
1:      int main(int argc, char *argv[]){  
2:          char buffer[10];  
3:          strcpy(buffer, argv[1]);  
4:          printf("%s\n", &buffer);  
5:      }
```

- 1 line : argc는 실행되는 프로그램의 인수 개수이며 \*argv[]는 포인터 배열로서 인자로 입력되는 값에 대한 번지수를 차례대로 저장함  
argv[0] - 실행 파일의 이름 / argv[1] : 첫번째 인자의 내용 / argv[2] : 두번째 인자의 내용
- 2 line : 10 byte 크기의 버퍼를 할당
- 3 line : 버퍼에 첫 번째 인자(argv[1])를 복사  
strcpy 함수는 입력된 인수의 경계를 체크하지 않으므로 10 byte가 넘는 인수를 받아도 스택에 쓰임

# [기초2] Buffer Overflow – Buffer Overflow 개요

## Buffer Overflow 분석 도구

### ■ gdb

| 명령              | 기능                                         |
|-----------------|--------------------------------------------|
| l (list)        | 디버깅하고자 하는 프로그램의 소스를 출력                     |
| b (break)       | 브레이크 포인트 설정                                |
| disass          | 프로그램의 특정 함수에 대한 어셈블리어 코드를 출력               |
| r (run)         | 디버깅하고자 하는 프로그램을 실행                         |
| print [변수]      | 변수 값을 출력                                   |
| info reg        | 레지스터에 저장된 값을 출력                            |
| w/[조회하는메모리범위]wx | 해당 주소 값에 해당하는 메모리 값 출력                     |
| s (step)        | 한 행씩 실행, 함수 포함하는 경우 함수 내부로 이동              |
| n (next)        | 한 행씩 실행, 함수 포함하는 경우 함수를 완전히 실행하고 다음 행으로 이동 |
| c (continue)    | 현재 위치에서 프로그램의 끝 또는 브레이크 포인트 까지 실행          |

# [기초2] Buffer Overflow – Stack Buffer Overflow

## Stack Buffer Overflow

### ■ Stack Buffer Overflow

- 보통 SetUID(Set User ID)가 설정된 루트 권한의 프로그램을 공격대상으로 함
- 스택에 정해진 버퍼보다 큰 공격 코드를 삽입하여 반환 주소를 변경함으로써 임의의 공격 코드를 루트 권한으로 실행하도록 하는 공격 방법

### ■ 함수 호출 매커니즘

- 호출된 함수가 끝났을 때 제어를 반환할 수 있도록 반환 주소를 저장할 장소가 필요
- 추가로 호출되는 함수에 전달할 인수와 호출된 함수가 반환될 때 계속 사용할 레지스터 값을 일반적으로 스택 프레임(stack frame)이라 알려진 구조체 안에 저장함

### ■ 셸 코드

- 오버플로우를 발생하여 실행 시키는 버퍼에 저장된 공격자의 코드

# [기초2] Buffer Overflow – Stack Buffer Overflow

## Stack Buffer Overflow

### ■ 공격 절차

- (1단계) : 공격 셀 코드를 버퍼에 저장
- (2단계) : 루트 권한으로 실행되는 프로그램의 특정 함수의 스택 반환주소 버퍼를 오버플로우시켜  
공격 셀 코드가 저장되어 있는 버퍼의 주소로 덮어씌움
- (3단계) : 특정 함수의 호출이 완료되면 조작된 반환 주소로 셀 코드의 주소가 반환되어 셀 코드가 실행되고  
루트 권한을 획득하게 됨

# [기초2] Buffer Overflow – Heap Buffer Overflow

## Heap Buffer Overflow

### ■ Heap Buffer Overflow

- Heap은 malloc 계열의 HeapAlloc, HeapFree, Malloc, free, new, delete 등 함수로 제어
- 일반적으로 Heap Buffer Overflow 공격을 통해 관리자 권한을 이용한 데이터 변조를 수행함
- Heap은 실행 영역이 아니며 실행 제어를 쉽게 이동시킬 수 있는 반환 주소도 없기 때문에 Stack Buffer Overflow처럼 셀을 바로 얻어내기 어려움

# [기초2] Buffer Overflow – Buffer Overflow 대응 방안

## Buffer Overflow 대응 방안

### ■ 안전한 함수 사용

- Buffer Overflow에 취약한 함수를 사용하지 않고 프로그램을 작성
- Buffer Overflow에 취약한 함수  
: strcat(), strcpy, getwd(), gets(), scanf(), sscanf(), vsprintf(), vsscanf(), sprintf(), vsprintf(),  
gethostbyname(), realpath()
- 사용을 권장하는 함수  
: strncpy(), fgets(), fscanf(), snprintf(), vsnprintf()

| 잘못된 strcpy 함수 사용                                                                                                 | 올바른 strncpy 함수 사용                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>void function(char *str){<br/>    char buffer[10];<br/>    strcpy(buffer, str);<br/>    return;<br/>}</pre> | <pre>void function(char *str){<br/>    char buffer[10];<br/>    strncpy(buffer, str, sizeof(buffer)-1);<br/>    buffer[sizeof(buffer)-1] = 0;<br/>    return;<br/>}</pre> |

# [기초2] Buffer Overflow – Buffer Overflow 대응 방안

## Buffer Overflow 대응 방안

### ■ Stack Guard

- gcc 컴파일러 확장 버전으로 Buffer Overflow를 탐지하도록 추가적인 함수 진입과 종료 코드를 삽입
- 컴파일러가 프로그램의 함수 호출(프로로그) 시에 ret 앞에 canary 값을 주입하고, 종료(return, 에필로그) 시에 canary 값이 변조되었는지의 여부를 확인하여 공격을 탐지

### ■ ASLR

- Address Space Layout Randomization
- 데이터 영역 주소를 난수화 하여 프로세스 주소 공간에 배치
- 공격자가 반환 주소를 조작할 때 주소를 예측하기 어렵게 만들기 위해 사용